

Files

📁 ..

📁 sample_data

📄 README.md

📄 anscombe.json

📄 california_housing_test.csv

📄 california_housing_train.csv

📄 mnist_test.csv

📄 mnist_train_small.csv

📄 shreenithy_spam_sms_model.pkl

📄 shreenithy_spam_tfidf_vectorizer...

📄 shreenithy_task2_model_compar...

📄 shreenithy_task2_predictions.csv

📄 spam.csv

Disk 87.30 GB available

[2] ✓ 5s

!pip install wordcloud

... Requirement already satisfied: wordcloud in /usr/local/lib/python3.12/dist-packages (1.9.6)
Requirement already satisfied: numpy>=1.19.3 in /usr/local/lib/python3.12/dist-packages (from wordcloud) (2.0.2)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from wordcloud) (11.3.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from wordcloud) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (26.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (2.9.0.p
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->wordcloud)

[4] ✓ 2m

import pandas as pd
import numpy as np
import re
import os
import joblib
import matplotlib.pyplot as plt

from wordcloud import WordCloud

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

Movie_Genre_Classification - Co

CodSoft_Task3_CustomerChurn

Spam_SMS_Detection.ipynb - Co

+

Ask Gemini

colab.research.google.com/drive/1YV-phUAZDS-xM6kIIWmrDj1r1Bkz1_6O#scrollTo=g1q_rP8-98am

☆

↓

👤

Action required

Spam_SMS_Detection.ipynb

☆

☁

File

Edit

View

Insert

Runtime

Tools

Help

🔍

Commands

+ Code

+ Text

▶ Run all

✓

RAM

Disk

⌵

⌶

Files

📁

✕

📁 ..

📁 sample_data

📄 README.md

📄 anscombe.json

📄 california_housing_test.csv

📄 california_housing_train.csv

📄 mnist_test.csv

📄 mnist_train_small.csv

📄 shreenithy_spam_sms_model.pkl

📄 shreenithy_spam_tfidf_vectorizer...

📄 shreenithy_task2_model_compar...

📄 shreenithy_task2_predictions.csv

📄 spam.csv

Disk

87.30 GB available

[4]

✓

2m

accuracy_score,
precision_score,
recall_score,
f1_score,
classification_report,
confusion_matrix,
ConfusionMatrixDisplay,
roc_curve,
auc
)

print("Uploaded files:")
print(os.listdir())
file_name = "spam.csv"
try:
 sms_data = pd.read_csv(file_name, encoding="latin-1")
except:
 sms_data = pd.read_csv(file_name, sep="\t", names=["Label", "Message"], encoding="latin-1")

print("\nDataset loaded successfully!")
print(sms_data.head())
print("\nOriginal Columns:")
print(sms_data.columns)

if "v1" in sms_data.columns and "v2" in sms_data.columns:
 sms_data = sms_data[["v1", "v2"]]
 sms_data.columns = ["Label", "Message"]

🔗

Variables

Terminal

Python 3

11:04
12-05-2026

Movie_Genre_Classification - Co

CodSoft_Task3_CustomerChurn

Spam_SMS_Detection.ipynb - Co

+

colab.research.google.com/drive/1YV-phUAZDS-xM6kIIWmrDj1r1Bkz1_6O#scrollTo=g1q_rP8-98am

Ask Gemini

☆

📄

📄

📄

Action required

🗨

⚙

👤 Share

👤

🔍 Commands

+ Code

+ Text

▶ Run all

✓ RAM

✓ Disk

Files

📁 ..

📁 sample_data

📄 README.md

📄 anscombe.json

📄 california_housing_test.csv

📄 california_housing_train.csv

📄 mnist_test.csv

📄 mnist_train_small.csv

📄 shreenithy_spam_sms_model.pkl

📄 shreenithy_spam_tfidf_vectorizer...

📄 shreenithy_task2_model_compar...

📄 shreenithy_task2_predictions.csv

📄 spam.csv

Disk

87.30 GB available

[4]

✓ 2m

```
elif len(sms_data.columns) >= 2:
    sms_data = sms_data.iloc[:, :2]
    sms_data.columns = ["Label", "Message"]

print("\nFinal Dataset:")
print(sms_data.head())

print("\nDataset Shape:", sms_data.shape)

print("\nMissing Values:")
print(sms_data.isnull().sum())

sms_data.dropna(inplace=True)

print("\nSpam and Ham Count:")
print(sms_data["Label"].value_counts())

sms_data["Message_Length"] = sms_data["Message"].apply(len)

print("\nAverage Message Length:")
print(sms_data.groupby("Label")["Message_Length"].mean())

plt.figure(figsize=(6, 4))
sms_data["Label"].value_counts().plot(kind="bar")
plt.title("Spam vs Ham SMS Count")
plt.xlabel("Message Type")
plt.ylabel("Count")
plt.show()

def clean_message(text):
```

Python 3

11:04

12-05-2026

Movie_Genre_Classification - Co

CodSoft_Task3_CustomerChurn

Spam_SMS_Detection.ipynb - Co

+

Ask Gemini

colab.research.google.com/drive/1YV-phUAZDS-xM6kIIWmrDj1r1Bkz1_6O#scrollTo=g1q_rP8-98am

☆

↓

Action required

Spam_SMS_Detection.ipynb

☆

☁

File

Edit

View

Insert

Runtime

Tools

Help

Q

Commands

+ Code

+ Text

▶ Run all

✓

RAM

Disk

Files

↑

↺

📁

🔒

..

sample_data

README.md

anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

shreenithy_spam_sms_model.pkl

shreenithy_spam_tfidf_vectorizer...

shreenithy_task2_model_compar...

shreenithy_task2_predictions.csv

spam.csv

Disk

87.30 GB available

[4]

✓ 2m

def clean_message(text):
 text = str(text).lower()
 text = re.sub(r"http\S+", "", text)
 text = re.sub(r"www\S+", "", text)
 text = re.sub(r"^[a-zA-Z\s]", "", text)
 text = re.sub(r"\s+", " ", text).strip()
 return text

sms_data["Cleaned_Message"] = sms_data["Message"].apply(clean_message)

print("\nOriginal Message Example:")
print(sms_data["Message"].iloc[0])

print("\nCleaned Message Example:")
print(sms_data["Cleaned_Message"].iloc[0])

sms_data["Label"] = sms_data["Label"].astype(str).str.lower()

sms_data["Encoded_Label"] = sms_data["Label"].map({
 "ham": 0,
 "spam": 1
})

sms_data.dropna(subset=["Encoded_Label"], inplace=True)

sms_data["Encoded_Label"] = sms_data["Encoded_Label"].astype(int)
X = sms_data["Cleaned_Message"]
y = sms_data["Encoded_Label"]

Variables

Terminal

Python 3

11:04

12-05-2026

Movie_Genre_Classification - Co

CodSoft_Task3_CustomerChurn

Spam_SMS_Detection.ipynb - Co

+

Ask Gemini

colab.research.google.com/drive/1YV-phUAZDS-xM6kIIWmrDj1r1Bkz1_6O#scrollTo=g1q_rP8-98am

☆

↓

👤

Action required

Spam_SMS_Detection.ipynb

☆

☁

File

Edit

View

Insert

Runtime

Tools

Help

Q

Commands

+ Code

+ Text

▶ Run all

✓

RAM

Disk

⌵

⌶

Files

↑ ↺ 📁 🔒

..

sample_data

README.md

anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

shreenithy_spam_sms_model.pkl

shreenithy_spam_tfidf_vectorizer...

shreenithy_task2_model_compar...

shreenithy_task2_predictions.csv

spam.csv

Disk

87.30 GB available

[4]

✓ 2m

X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42,
stratify=y
)

print("\nTraining Size:", X_train.shape[0])
print("Testing Size:", X_test.shape[0])

tfidf_vectorizer = TfidfVectorizer(
stop_words="english",
max_features=8000,
ngram_range=(1, 2),
min_df=1
)

X_train_vector = tfidf_vectorizer.fit_transform(X_train)
X_test_vector = tfidf_vectorizer.transform(X_test)

print("\nTF-IDF Vectorization completed!")
print("Vector Shape:", X_train_vector.shape)

models = {
"Naive Bayes": MultinomialNB(),
"Logistic Regression": LogisticRegression(max_iter=2000),
"Linear SVM": LinearSVC(),
"Random Forest": RandomForestClassifier(n_estimators=200, random_state=42)

{}

Variables

Terminal

Python 3

11:04

12-05-2026



▼ sample_data

 README.md

california_housing_test.csv

california_housing_train.csv

CSV mnist test.csv

```
CSV mnist train small.csv
```

shreenithy_spam_sms_model.pkl

shreenithy_spam_tfidf_vectorizer...

CSV shreenithy task2 model compar...

shreenithy task2 predictions.csv

 spam.csv

Disk 87.30 GB available

[4] ✓ 2m



```
comparison_df = pd.DataFrame(results).T
```

```
print("\n===== MODEL COMPARISON =====")
print(comparison_df)
```

```
comparison df.to_csv("shreenithy task2 model comparison.csv")
```

```
best_model_name = comparison_df["Accuracy"].idxmax()
best_model = models[best_model_name]
best_prediction = predictions[best_model_name]
```

```
print("\nBest Model:", best_model_name)
print("Best Accuracy:", comparison_df.loc[best_model_name, "Accuracy"])
plt.figure(figsize=(8, 5))
plt.bar(comparison_df.index, comparison_df["Accuracy"])
plt.title("Spam SMS Detection - Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.xticks(rotation=20)
plt.show()
comparison_df[["Precision", "Recall", "F1 Score"]].plot(kind="bar", figsize=(9, 5))
plt.title("Precision, Recall and F1 Score Comparison")
plt.xlabel("Models")
plt.ylabel("Score")
plt.xticks(rotation=20)
plt.show()
```

```
cm = confusion matrix(y test, best prediction)
```

```
disn = ConfusionMatrixDisnlay(
```

 Variables Terminal

 Python 3

Files

- ..
- sample_data
 - README.md
 - anscombe.json
 - california_housing_test.csv
 - california_housing_train.csv
 - mnist_test.csv
 - mnist_train_small.csv
- shreenithy_spam_sms_model.pkl
- shreenithy_spam_tfidf_vectorizer...
- shreenithy_task2_model_compar...
- shreenithy_task2_predictions.csv
- spam.csv

Disk 87.30 GB available

```
[4] ✓ 2m  
)  
  
disp.plot()  
plt.title("Confusion Matrix - Best Spam Detection Model")  
plt.show()  
  
if best_model_name == "Linear SVM":  
    score_values = best_model.decision_function(X_test_vector)  
else:  
    score_values = best_model.predict_proba(X_test_vector)[:, 1]  
  
fpr, tpr, threshold = roc_curve(y_test, score_values)  
roc_auc = auc(fpr, tpr)  
  
plt.figure(figsize=(6, 5))  
plt.plot(fpr, tpr, label="AUC = %.2f" % roc_auc)  
plt.plot([0, 1], [0, 1], linestyle="--")  
plt.xlabel("False Positive Rate")  
plt.ylabel("True Positive Rate")  
plt.title("ROC Curve - Spam SMS Detection")  
plt.legend()  
plt.show()  
spam_words = " ".join(sms_data[sms_data["Encoded_Label"] == 1]["Cleaned_Message"])  
ham_words = " ".join(sms_data[sms_data["Encoded_Label"] == 0]["Cleaned_Message"])  
  
spam_wordcloud = WordCloud(  
    width=800,  
    height=400,  
    background_color="white"  
)  
generate(spam_words)
```


Files

- ..
- sample_data
 - README.md
 - anscombe.json
 - california_housing_test.csv
 - california_housing_train.csv
 - mnist_test.csv
 - mnist_train_small.csv
- shreenithy_spam_sms_model.pkl
- shreenithy_spam_tfidf_vectorizer...
- shreenithy_task2_model_compar...
- shreenithy_task2_predictions.csv
- spam.csv

Disk 87.30 GB available

```
[4] ✓ 2m
plt.figure(figsize=(10, 5))
plt.imshow(spam_wordcloud)
plt.axis("off")
plt.title("Most Common Words in Spam Messages")
plt.show()

ham_wordcloud = WordCloud(
    width=800,
    height=400,
    background_color="white"
).generate(ham_words)

plt.figure(figsize=(10, 5))
plt.imshow(ham_wordcloud)
plt.axis("off")
plt.title("Most Common Words in Ham Messages")
plt.show()

prediction_df = pd.DataFrame({
    "Actual_Label": y_test.values,
    "Predicted_Label": best_prediction
})

prediction_df["Actual_Label"] = prediction_df["Actual_Label"].map({
    0: "Ham",
    1: "Spam"
})

prediction_df["Predicted_Label"] = prediction_df["Predicted_Label"].map({
    0: "Ham",
```


Files

📁 ..

📁 sample_data

📄 README.md

📄 anscombe.json

📄 california_housing_test.csv

📄 california_housing_train.csv

📄 mnist_test.csv

📄 mnist_train_small.csv

📄 shreenithy_spam_sms_model.pkl

📄 shreenithy_spam_tfidf_vectorizer...

📄 shreenithy_task2_model_compar...

📄 shreenithy_task2_predictions.csv

📄 spam.csv

Disk 87.30 GB available

```
Uploaded files:
['.config', 'shreenithy_task2_model_comparison.csv', 'shreenithy_task2_predictions.csv', 'shreenithy_spam_sms_model.pkl', 'shreenithy_spam_tfidf_vectorizer.pkl']

Dataset loaded successfully!

   v1                                     v2 Unnamed: 2 \
0  ham  Go until jurong point, crazy.. Available only ...      NaN
1  ham                                     Ok lar... Joking wif u oni...      NaN
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...      NaN
3  ham  U dun say so early hor... U c already then say...      NaN
4  ham  Nah I don't think he goes to usf, he lives aro...      NaN

   Unnamed: 3 Unnamed: 4
0      NaN      NaN
1      NaN      NaN
2      NaN      NaN
3      NaN      NaN
4      NaN      NaN

Original Columns:
Index(['v1', 'v2', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], dtype='object')

Final Dataset:
   Label                                     Message
0  ham  Go until jurong point, crazy.. Available only ...
1  ham                                     Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3  ham  U dun say so early hor... U c already then say...
4  ham  Nah I don't think he goes to usf, he lives aro...

Dataset Shape: (5572, 2)

Missing Values:
```


Files

📁 ..

📁 sample_data

📄 README.md

📄 anscombe.json

📄 california_housing_test.csv

📄 california_housing_train.csv

📄 mnist_test.csv

📄 mnist_train_small.csv

📄 shreenithy_spam_sms_model.pkl

📄 shreenithy_spam_tfidf_vectorizer...

📄 shreenithy_task2_model_compar...

📄 shreenithy_task2_predictions.csv

📄 spam.csv

Disk 87.30 GB available

Final Dataset:

	Label	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Dataset Shape: (5572, 2)

Missing Values:

Label	0
Message	0

dtype: int64

Spam and Ham Count:

Label	
ham	4825
spam	747

Name: count, dtype: int64

Average Message Length:

Label	
ham	71.023627
spam	138.866131

Name: Message_Length, dtype: float64

Spam vs Ham SMS Count

Label	Count
ham	4825
spam	747

Movie_Genre_Classification - Co xCodSoft_Task3_CustomerChurn xSpam_SMS_Detection.ipynb - Co x

colab.research.google.com/drive/1YV-phUAZDS-xM6kIIWmrDj1r1Bkz1_6O#scrollTo=g1q_rP8-98am

Ask Gemini

Action required

Spam_SMS_Detection.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAMDisk

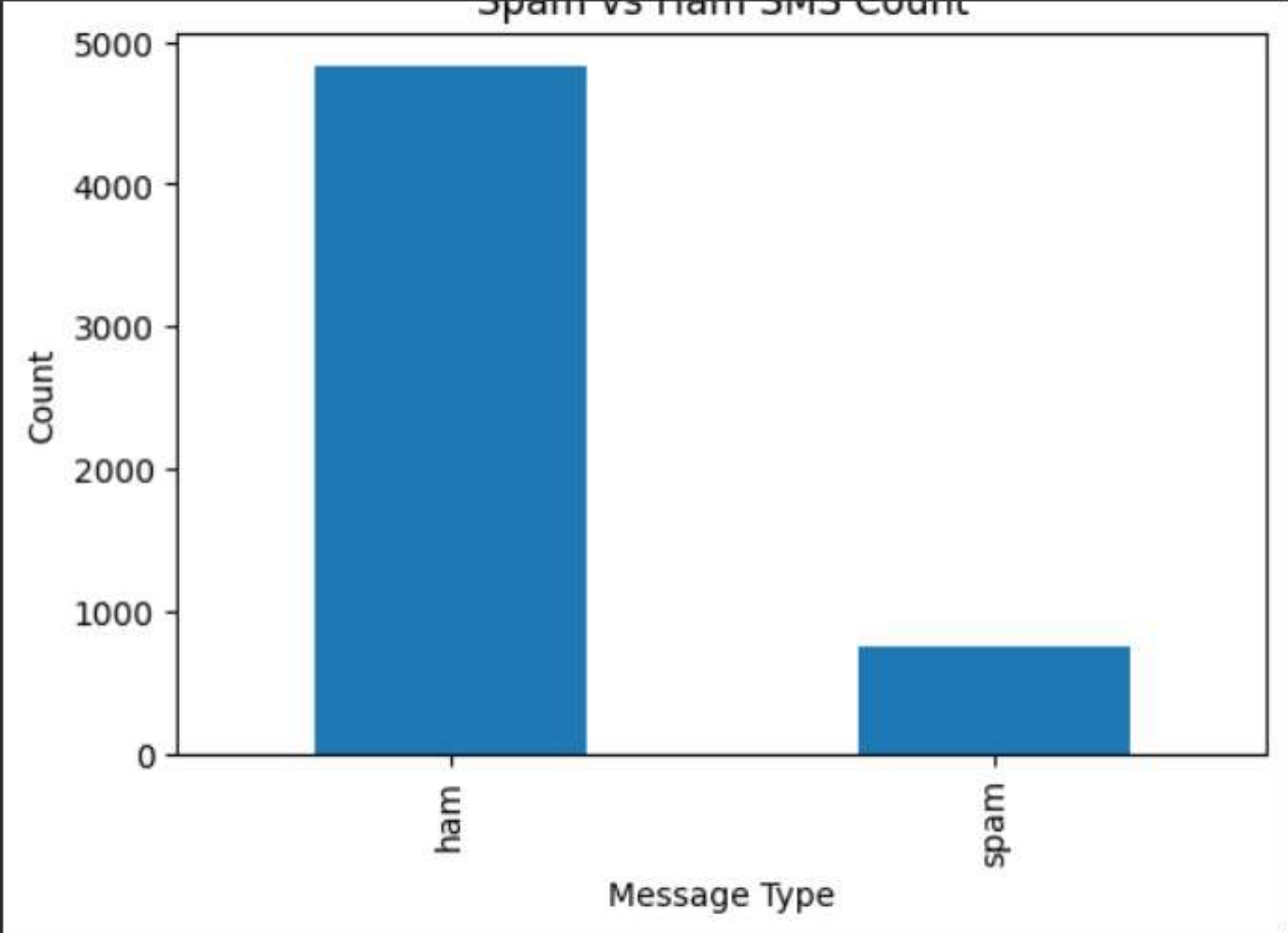
Files

sample_data

README.mdanscombe.jsoncalifornia_housing_test.csvcalifornia_housing_train.csvmnist_test.csvmnist_train_small.csvshreenithy_spam_sms_model.pklshreenithy_spam_tfidf_vectorizer...shreenithy_task2_model_compar...shreenithy_task2_predictions.csvspam.csv

Disk87.30 GB available

Spam vs Ham SMS Count



Message Type	Count
ham	4857
spam	777

Original Message Example:

Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...

Cleaned Message Example:

go until jurong point crazy available only in bugis n great world la e buffet cine there got amore wat

Training Size: 4457

Testing Size: 1115

Variables

Terminal

Python 3

⋮

🔍

⏪

👁

🔑

📁

📊

Files

📁 ..

📁 sample_data

📄 README.md

📄 anscombe.json

📄 california_housing_test.csv

📄 california_housing_train.csv

📄 mnist_test.csv

📄 mnist_train_small.csv

📄 shreenithy_spam_sms_model.pkl

📄 shreenithy_spam_tfidf_vectorizer...

📄 shreenithy_task2_model_compar...

📄 shreenithy_task2_predictions.csv

📄 spam.csv

Disk 87.30 GB available

```
Training Size: 4457
Testing Size: 1115

...

TF-IDF Vectorization completed!
Vector Shape: (4457, 8000)

=====
Training Model: Naive Bayes
=====
Accuracy: 0.9641255605381166
Precision: 0.990990990990991
Recall: 0.738255033557047
F1 Score: 0.8461538461538461

Classification Report:
              precision    recall  f1-score   support

      Ham      0.96      1.00      0.98        966
      Spam      0.99      0.74      0.85        149

   accuracy      0.96      0.96      0.96      1115
  macro avg      0.98      0.87      0.91      1115
weighted avg      0.97      0.96      0.96      1115

=====
Training Model: Logistic Regression
=====
Accuracy: 0.9668161434977578
Precision: 0.9912280701754386
Recall: 0.7583892617449665
F1 Score: 0.8593155893536122
```


Files

📁

✕

📁 ..

📁 sample_data

📄 README.md

📄 anscombe.json

📄 california_housing_test.csv

📄 california_housing_train.csv

📄 mnist_test.csv

📄 mnist_train_small.csv

📄 shreenithy_spam_sms_model.pkl

📄 shreenithy_spam_tfidf_vectorizer...

📄 shreenithy_task2_model_compar...

📄 shreenithy_task2_predictions.csv

📄 spam.csv

Disk

87.30 GB available

```
Classification Report:
...
              precision    recall  f1-score   support

      Ham      0.96      1.00      0.98       966
      Spam      0.99      0.76      0.86       149

   accuracy      0.97      0.97      0.96      1115
  macro avg      0.98      0.88      0.92      1115
 weighted avg      0.97      0.97      0.96      1115

=====
Training Model: Linear SVM
=====
Accuracy: 0.9847533632286996
Precision: 0.9852941176470589
Recall: 0.8993288590604027
F1 Score: 0.9403508771929825

Classification Report:
              precision    recall  f1-score   support

      Ham      0.98      1.00      0.99       966
      Spam      0.99      0.90      0.94       149

   accuracy      0.98      0.95      0.98      1115
  macro avg      0.98      0.95      0.97      1115
 weighted avg      0.98      0.98      0.98      1115

=====
```


Files

📁 ..

📁 sample_data

📄 README.md

📄 anscombe.json

📄 california_housing_test.csv

📄 california_housing_train.csv

📄 mnist_test.csv

📄 mnist_train_small.csv

📄 shreenithy_spam_sms_model.pkl

📄 shreenithy_spam_tfidf_vectorizer...

📄 shreenithy_task2_model_compar...

📄 shreenithy_task2_predictions.csv

📄 spam.csv

Disk 87.30 GB available

Training Model: Random Forest

=====

Accuracy: 0.9757847533632287

Precision: 1.0

Recall: 0.8187919463087249

F1 Score: 0.9003690036900369

Classification Report:

	precision	recall	f1-score	support
Ham	0.97	1.00	0.99	966
Spam	1.00	0.82	0.90	149
accuracy			0.98	1115
macro avg	0.99	0.91	0.94	1115
weighted avg	0.98	0.98	0.97	1115

===== MODEL COMPARISON =====

	Accuracy	Precision	Recall	F1 Score
Naive Bayes	0.964126	0.990991	0.738255	0.846154
Logistic Regression	0.966816	0.991228	0.758389	0.859316
Linear SVM	0.984753	0.985294	0.899329	0.940351
Random Forest	0.975785	1.000000	0.818792	0.900369

Best Model: Linear SVM

Best Accuracy: 0.9847533632286996

Spam SMS Detection - Accuracy Comparison

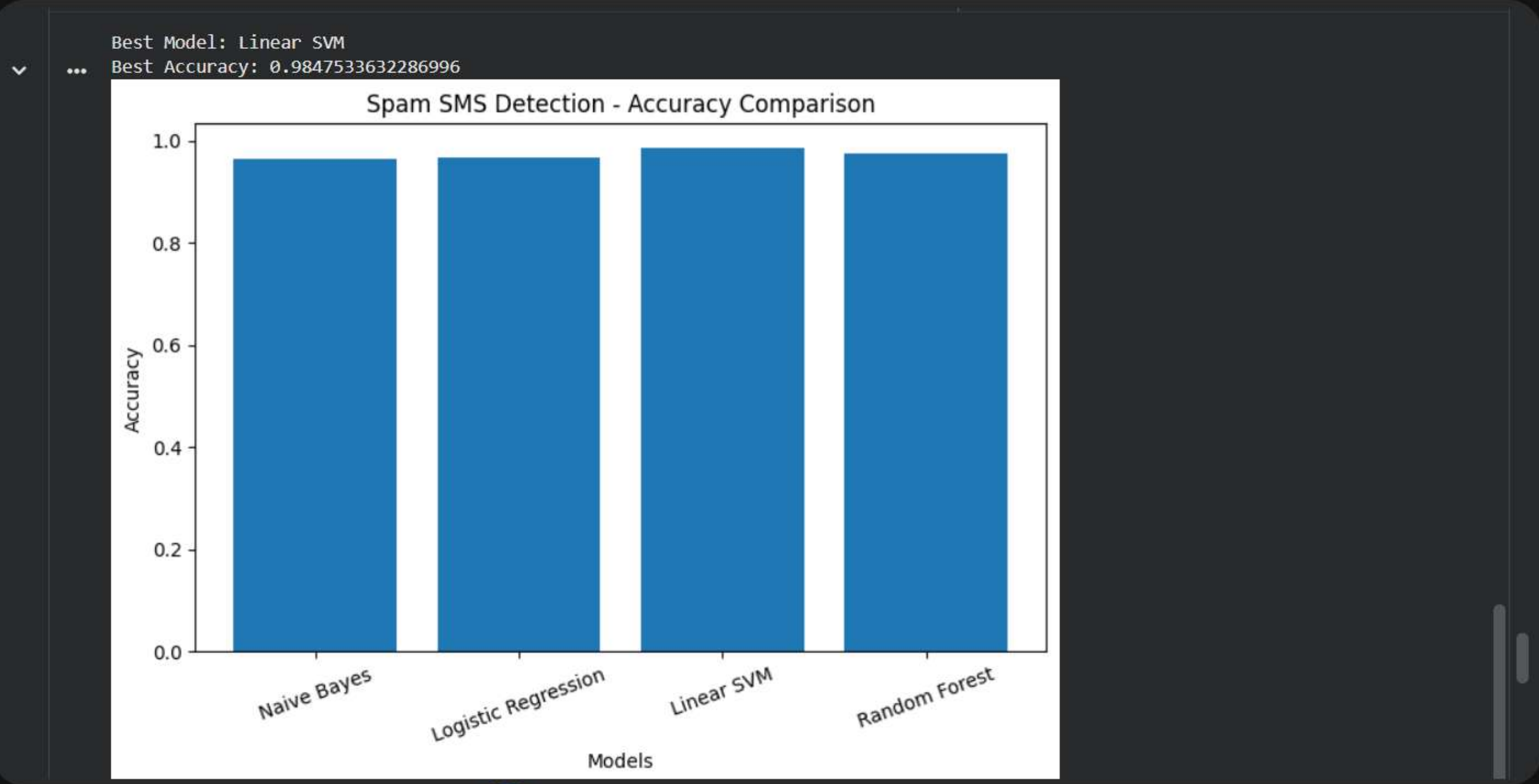


Model	Accuracy
Naive Bayes	0.964126
Logistic Regression	0.966816
Linear SVM	0.984753
Random Forest	0.975785

Files

- ..
- sample_data
 - README.md
 - anscombe.json
 - california_housing_test.csv
 - california_housing_train.csv
 - mnist_test.csv
 - mnist_train_small.csv
- shreenithy_spam_sms_model.pkl
- shreenithy_spam_tfidf_vectorizer...
- shreenithy_task2_model_compar...
- shreenithy_task2_predictions.csv
- spam.csv

Disk 87.30 GB available



Files

📁 ..

📁 sample_data

📄 README.md

📄 anscombe.json

📄 california_housing_test.csv

📄 california_housing_train.csv

📄 mnist_test.csv

📄 mnist_train_small.csv

📄 shreenithy_spam_sms_model.pkl

📄 shreenithy_spam_tfidf_vectorizer...

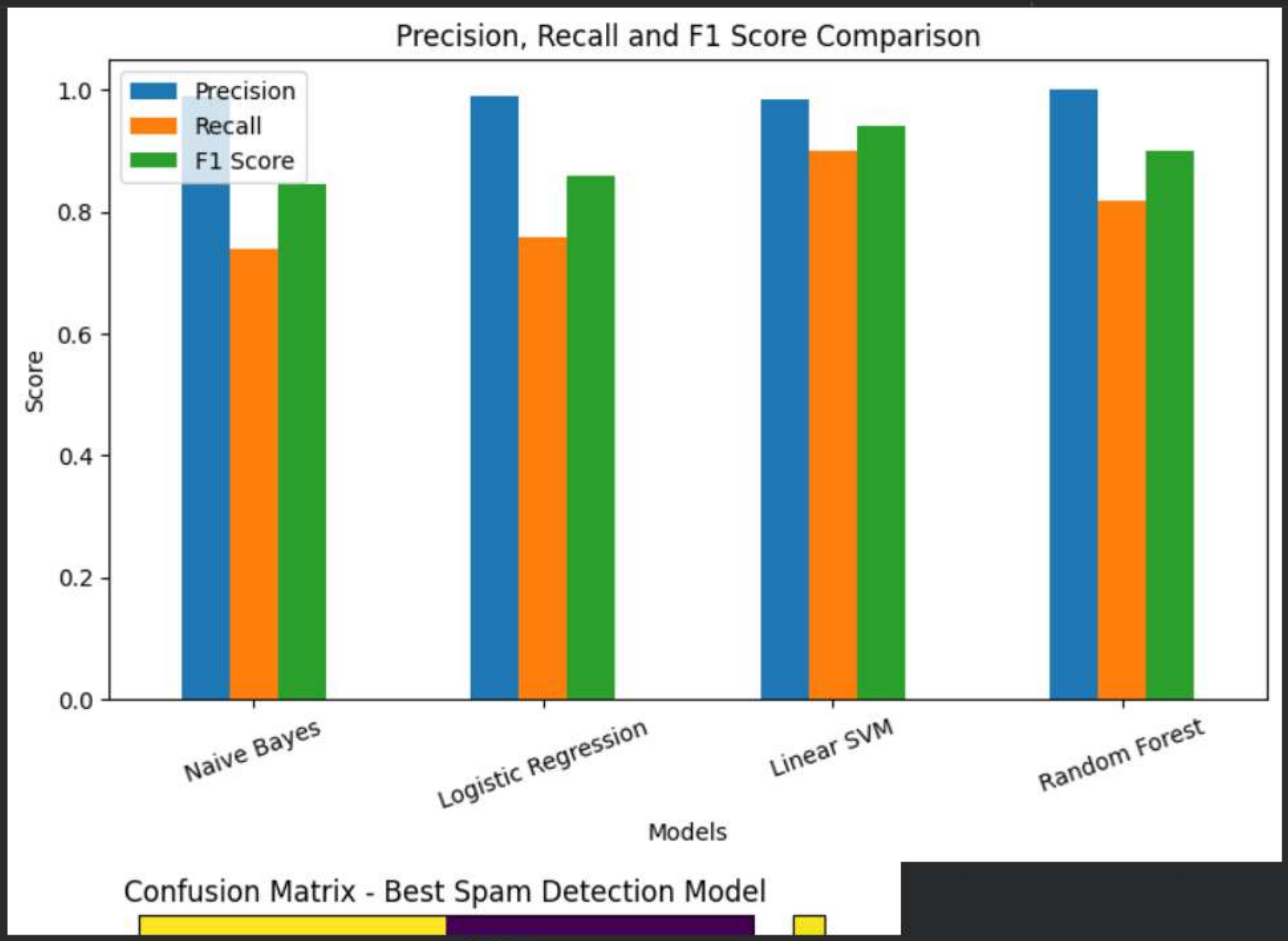
📄 shreenithy_task2_model_compar...

📄 shreenithy_task2_predictions.csv

📄 spam.csv

Disk

87.30 GB available



Movie_Genre_Classification - Co xCodSoft_Task3_CustomerChurnl xSpam_SMS_Detection.ipynb - C x

colab.research.google.com/drive/1YV-phUAZDS-xM6klIWmrDj1r1Bkz1_6O#scrollTo=g1q_rP8-98am

Ask Gemini

Action required

Spam_SMS_Detection.ipynb

FileEditViewInsertRuntimeToolsHelp

CommandsCodeTextRun all

RAMDisk

Files

sample_data

README.mdanscombe.jsoncalifornia_housing_test.csvcalifornia_housing_train.csvmnist_test.csvmnist_train_small.csvshreenithy_spam_sms_model.pklishreenithy_spam_tfidf_vectorizer...shreenithy_task2_model_compar...shreenithy_task2_predictions.csvspam.csv

Disk87.30 GB available

Confusion Matrix - Best Spam Detection Model

Ham	964	2
Spam	15	134

ROC Curve - Spam SMS Detection

VariablesTerminal

Python 3

  ..

sample_data

 README.md

JSON `anscombe.json`

CSV california_housing_test.csv

california_housing_train.csv

CSV mnist_test.csv

```
CSV mnist_train_small.csv
```

shreenithy_spam_sms_model.pkl

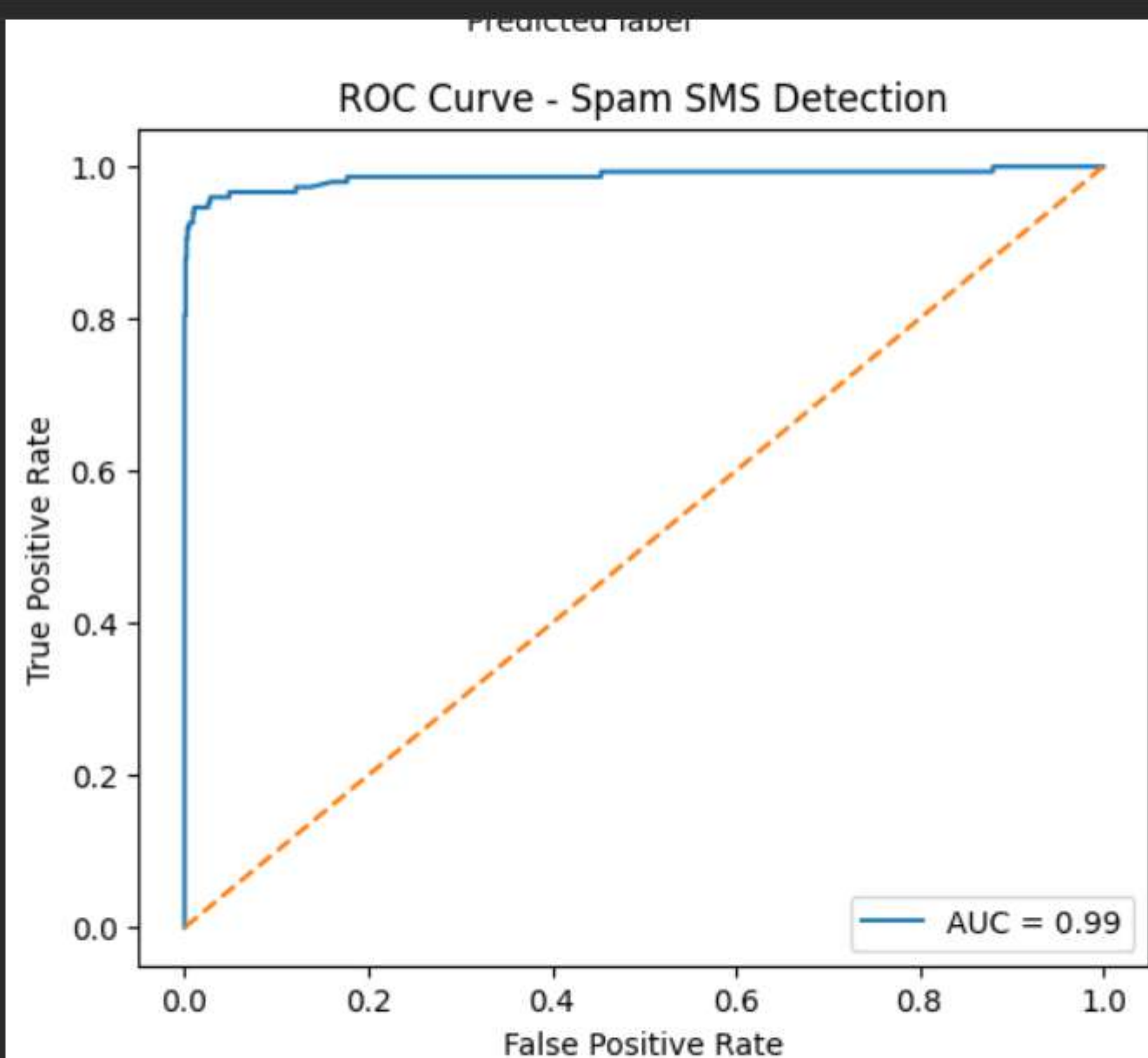
shreenithy_spam_tfidf_vectorizer...

CSV shreenithy_task2_model_compar...

shreenithy_task2_predictions.csv

CSV spam.csv

Disk 87.30 GB available



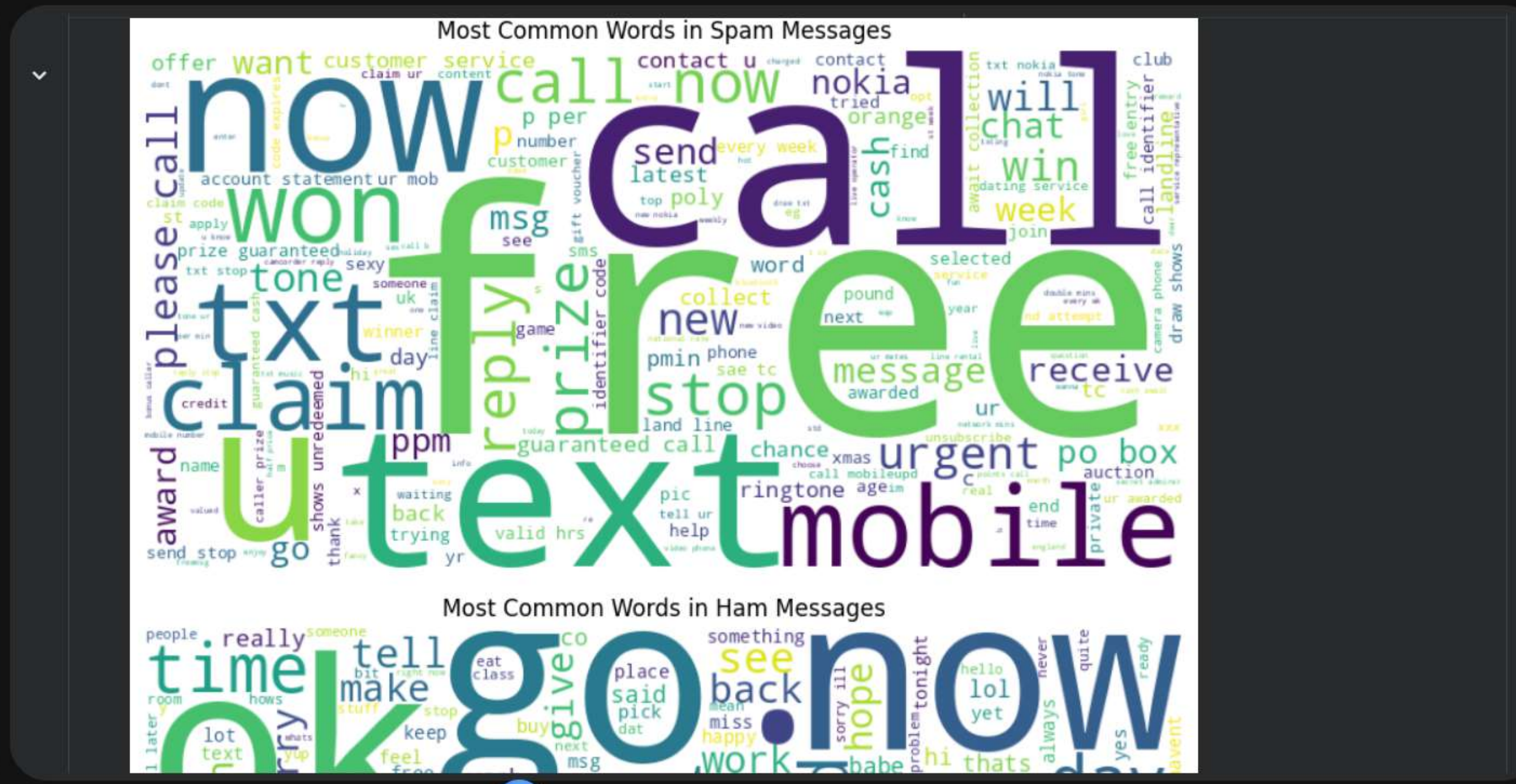
Most Common Words in Spam Messages

offer want customer service contact u changed contact nokia 1 txt nokia 1 club
claim ur content start now nokia 1 action will try

Files

- ..
- sample_data
 - README.md
 - anscombe.json
 - california_housing_test.csv
 - california_housing_train.csv
 - mnist_test.csv
 - mnist_train_small.csv
- shreenithy_spam_sms_model.pkl
- shreenithy_spam_tfidf_vectorizer...
- shreenithy_task2_model_compar...
- shreenithy_task2_predictions.csv
- spam.csv

Disk 87.30 GB available



Movie_Genre_Classification - CoCodSoft_Task3_CustomerChurnSpam_SMS_Detection.ipynb

colab.research.google.com/drive/1YV-phUAZDS-xM6kIIWmrDj1r1Bkz1_6O#scrollTo=g1q_rP8-98am

Ask Gemini

Action required

Spam_SMS_Detection.ipynb

FileEditViewInsertRuntimeToolsHelp

CommandsCodeTextRun all

RAMDisk

Files

..

sample_data

README.md

anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

shreenithy_spam_sms_model.pkl

shreenithy_spam_tfidf_vectorizer...

shreenithy_task2_model_compar...

shreenithy_task2_predictions.csv

spam.csv

Disk87.30 GB available

Prediction CSV saved successfully!

...===== SAMPLE SMS PREDICTIONS =====

SMS: Congratulations! You have won a free iPhone. Click the link now.
Prediction: Spam Message

SMS: Hey, are you coming to college tomorrow?
Prediction: Ham Message

SMS: You won 50000 rupees cash prize. Claim immediately.
Prediction: Spam Message

SMS: Your OTP is 4589. Do not share it with anyone.
Prediction: Ham Message

SMS: Free recharge offer available only today. Click here.
Prediction: Ham Message

SMS: Call me when you reach home.
Prediction: Ham Message

SMS: URGENT! Your account has been blocked. Verify now.
Prediction: Spam Message

SMS: Good morning, submit your assignment today.
Prediction: Ham Message

Enter any SMS message to check Spam or Ham: Call me when you reach home.

Your SMS:
Call me when you reach home.

VariablesTerminal

Python 3

Movie_Genre_Classification - Co xCodSoft_Task3_CustomerChurn xSpam_SMS_Detection.ipynb - Co x

colab.research.google.com/drive/1YV-phUAZDS-xM6kIIWmrDj1r1Bkz1_6O#scrollTo=g1q_rP8-98am

Ask Gemini

Action required

Spam_SMS_Detection.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAMDisk

Files

..

sample_data

README.md

anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

shreenithy_spam_sms_model.pkl

shreenithy_spam_tfidf_vectorizer...

shreenithy_task2_model_compar...

shreenithy_task2_predictions.csv

spam.csv

Disk87.30 GB available

...

Your SMS:
Call me when you reach home.

Prediction Result:
Ham Message

Model and vectorizer saved successfully!

===== TASK 2 COMPLETED =====

Project Name: Spam SMS Detection

Best Model: Linear SVM

Best Accuracy: 0.9847533632286996

Models Used:

- Naive Bayes

- Logistic Regression

- Linear SVM

- Random Forest

Task 2 completed successfully!

Double-click (or enter) to edit

Variables

Terminal

Python 3